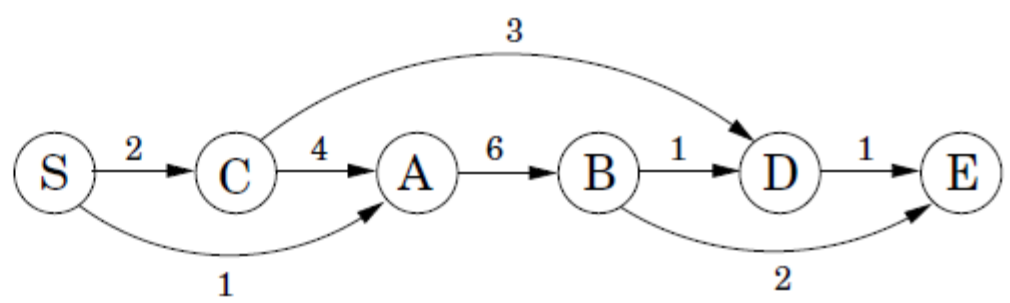
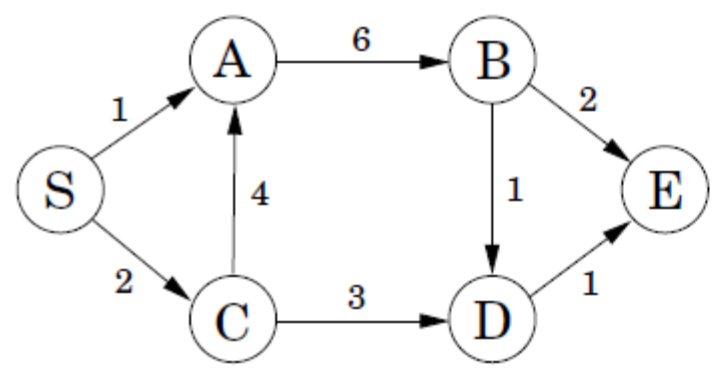


Dynamic Programming

Shortest Path in DAG (from starting node)



DAG structure, can be solved by recursion!

We were considering vertices (u) in linearized order then applying the update

```
procedure update( $(u,v) \in E$ )  
   $\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u,v)\}$ 
```

Example:

$$\text{dist}(D) = \min\{\text{dist}(B) + 1, \text{dist}(C) + 3\}$$

initialize all $\text{dist}(\cdot)$ values to ∞

Bookkeeping $\text{dist}(s) = 0$

to get path

for each $v \in V \setminus \{s\}$, in linearized order:

$$\text{dist}(v) = \min_{(u,v) \in E} \{\text{dist}(u) + l(u,v)\}$$

Iteration?!

Till the node needed!

S, C, A, B, D, E – 0, 1, 2, 3, 4, 5

→ Need implicit DAG to avoid infinite recursion

S / 0 – starting node

Careful: m & n do not denote # of edges & nodes here

```
function Sspdag(n)  Sspdag(G)?  $\forall v \in V, \text{Visi}[v]$   
  create an array dist[0 ... |V| - 1] with all values  $\infty$   
  return Sspdagmemo(n, dist)
```

```
function Sspdagmemo(n, d)
```

```
  if  $n = 0$ : return 0
```

```
  elif  $d[n] \neq \infty$ : return  $d[n]$ 
```

```
  elif in-degree( $n$ ) = 0: return  $\infty$ 
```

```
  else:
```

```
     $\forall (m, n)$ :
```

```
       $d[n] = \min(d[n], l(m, n) + \text{Sspdagmemo}(m, d))$ 
```

```
  return  $d[n]$ 
```

Can use the adjacency list of the reverse graph

Complexity? Topological sort – linear, Reverse graph – linear so In-degree – linear, edge presence – linear ($|V| + |E|$), loop over all edges – linear (lookup, add, min) ($|E|$)

→ Storing the node to be printed

Time - $|V| + |E|$, memory for memoization: $|V|$

Dynamic programming is a very powerful algorithmic paradigm in which a problem is solved by identifying a collection of subproblems and tackling them one by one, “smallest” first, using the answers to “small” problems to help figure out “larger” ones, until the whole lot of them is solved.



*Conceptually DAG
is implicit*

The path...

*May not be Greedy because a local
(subproblem) optimal computation may
not be part of the final solution!*

```
function Sspdag(n)
create an array dist[0 ... |V| - 1] with all values ∞
create a list prev[0 ... |V| - 1] with all empty lists [ ]
D, P = Sspdagmemo(n, dist d, prev p)
return D, P
```

```
function Sspdagmemo(n, d, p)
if n = 0: return 0, [n]
elif d[n] ≠ ∞: return d[n], p[n]
elif in-degree(n) = 0: return ∞, [ ]
else:
  ∀(m, n):
    δ, ρ = Sspdagmemo(m, d, p)
    v = l(m, n) + δ
    d[n] = min(d[n], v)
    if d[n] == v:
      p[n] = p[m].append(m)
return d[n], p[n]
```

*We can also find:
Longest path,
smallest product
(instead of sum)*

Space: $|V|^2$

Longest Increasing Subsequence

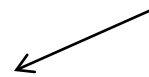
Sequence $a_1, \dots, a_n$



Subsequence $a_{i_1}, a_{i_2}, \dots, a_{i_k}$

$$1 \leq i_1 < i_2 < \dots < i_k \leq n$$

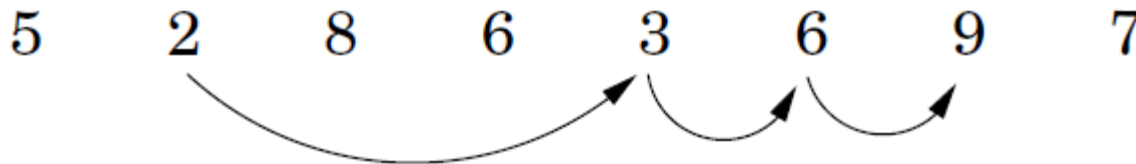
Increasing subsequence:



$$a_{i_1} < a_{i_2} < \dots < a_{i_k}$$

Find: increasing subsequence of greatest length

Example.. 5, 2, 8, 6, 3, 6, 9, 7



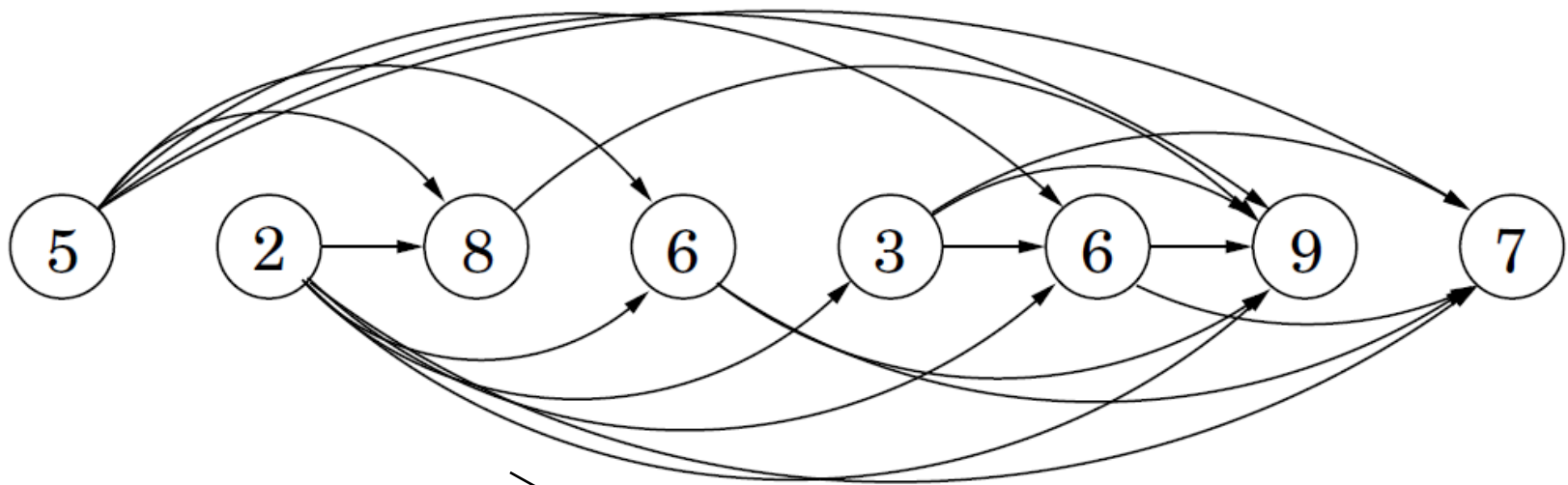
$\Rightarrow$ 2, 3, 6, 9

Lets build a graph:

- Consider each number a_i in the sequence as a node
- Starting from the first (leftmost) node:

Add a directed edge (i, j) whenever $i < j$ & $a_i < a_j$

whenever it is possible for a_i and a_j to be consecutive elements in an increasing subsequence.



Time complexity:

$$\frac{n(n-1)}{2}$$

$n \rightarrow |V|$

$$G = (V, E)$$

Space: $O(m)$

will be a **DAG** as for every edge (i, j) we have $i < j$.

All paths in the graph G will represent increasing subsequences!

All edges have the same weight, so find the longest path in G !

Top-down / Bottom-up using DAG ($|V|^2$):

-ive of edge weights

Shortest path in DAG $\rightarrow$ Longest path in DAG

But we need to compute longest path from every node to every node!

Memory for memoization: $c|V| + 1$

*Storing 'max' dist
& its path is extra*

To get this

TD Time complexity: $(|V| + |E|)|V|$

'Max' from this!

BU Time complexity: $|V|(|V| + |E|)$

$|E|$ (graph space)

Another DAG

Bottom-up:

0 initialization

for $j = 1, 2, \dots, n$:

The order in sequence
(already linearized!)

Length of
the
longest
path to j

$L(j) = 1 + \max\{L(i) : (i, j) \in E\}$

$|V|^2$ (comp),

return $\max_j L(j)$

$|E|$ (space)

Bookkeeping

End

*Because,
every node can't
be a starter!*

Maximum of the longest paths!
(Longest increasing subsequence)

Memory: Linear in $|V|$

Time: $|E| + |V|$ (without graph building)

Inverse graph is linear, max L is linear, Within loop constant cost is applied times linear in edges.

In the top-down approach, initialize with '0' and use max, just like bottom-up! The complexity will be linear, space is linear!

**There is a $O(n \log n)$ solution available outside DP /hybrid DP*

Another Dynamic!

Problem: *largest $L(j)$*

subproblems $\{L(j) : 1 \leq j \leq n\}$

(*) There is an ordering on the subproblems, and a relation that shows how to solve a subproblem given the answers to “smaller” subproblems, that is, subproblems that appear earlier in the ordering.

the relation $\longrightarrow L(j) = 1 + \max\{L(i) : (i, j) \in E\}$

Same complexity $O(n^2)$, NO DAG!

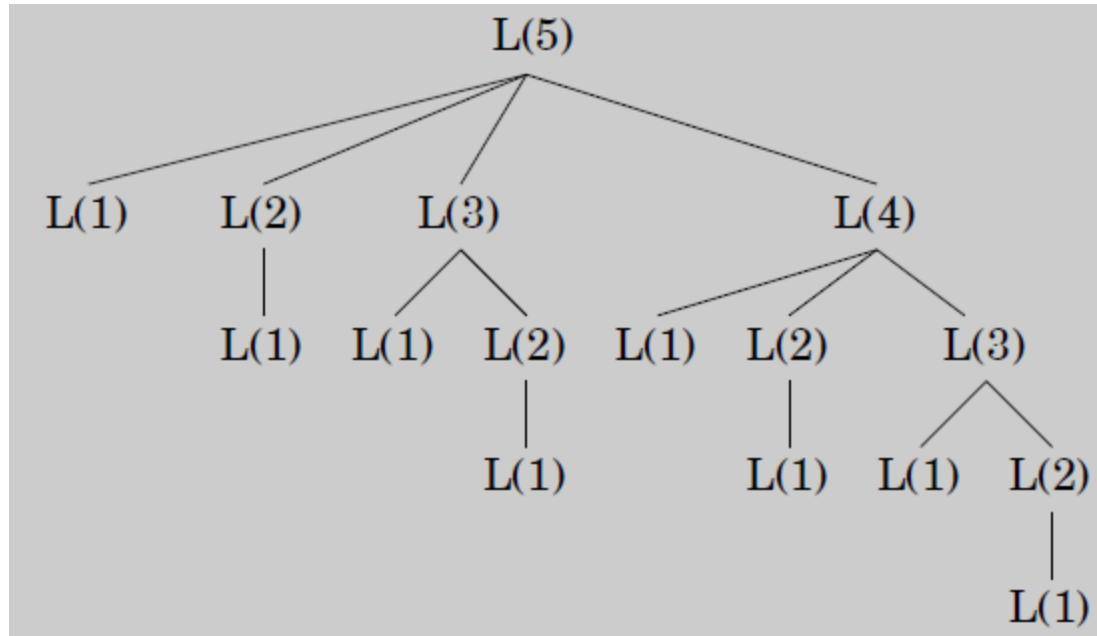
Space - $O(n)$

$\searrow$
 $\longrightarrow$ or loop: $\forall i < j, \text{ where } a_i < a_j$

Worst implementation:

the dag contains edges (i, j) for *all* $i < j$

No memory used for storing values (ignoring graph), but exponential complexity!



Redundant computations!

You need to compute again & again the same stuff!

Preferred implementation: *Time* — $O(n \log n)$, *Space* — $O(n)$

Involves binary search

Edit Distance

Input: Two strings $x[1 \dots m], y[1 \dots n]$

Task: Compute fewest number of edits to convert x into y

Operations allowed:

Insert char into x , Delete char from x , substitute a char for another in x

S — N O W Y

S U N N — Y

Cost: 3

— S N O W — Y

S U N — — N Y

Cost: 5

A DP solution

some *prefix* of the first string, $x[1 \dots i]$

prefix of the second, $y[1 \dots j]$

edit distance $\longrightarrow$ subproblem $E(i, j)$

final objective $\longrightarrow E(m, n)$

$E(7, 5)$

E	X	P	O	N	E	N
---	---	---	---	---	---	---

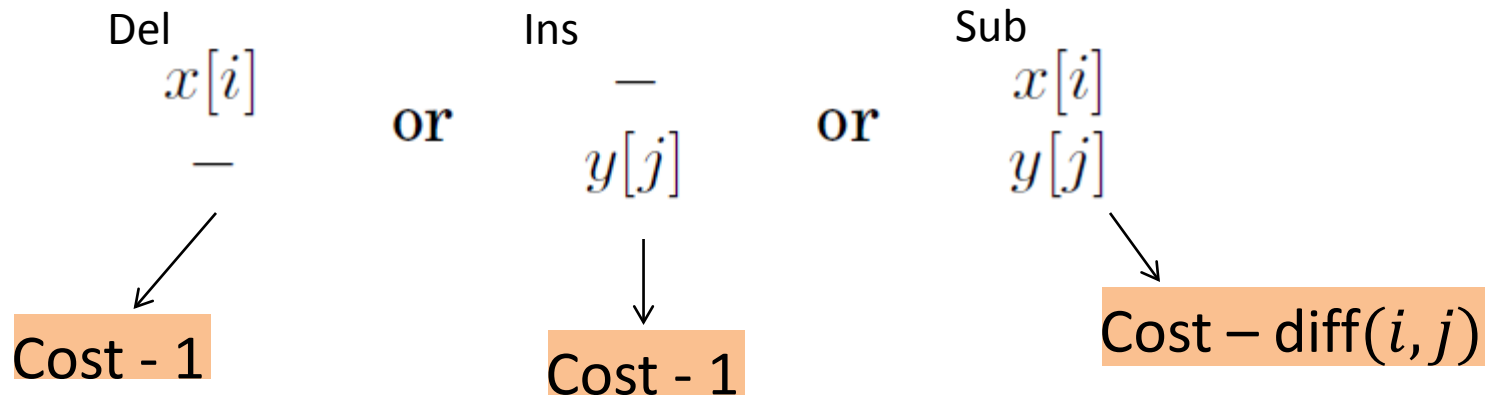
 T I A L

P	O	L	Y	N
---	---	---	---	---

 O M I A L

best alignment between $x[1 \dots i]$ and $y[1 \dots j]$?

The following edits are possible among the right most entities to match (change) the strings:



Prev (next):

$$\begin{array}{ccc} x[1 \dots i-1] \leftrightarrow y[1 \dots j] & x[1 \dots i] \leftrightarrow y[1 \dots j-1] & x[1 \dots i-1] \leftrightarrow y[1 \dots j-1] \\ E(i-1, j) & E(i, j-1) & E(i-1, j-1) \end{array}$$

$$E(i, j) = \min\{1 + E(i-1, j), 1 + E(i, j-1), \text{diff}(i, j) + E(i-1, j-1)\}$$

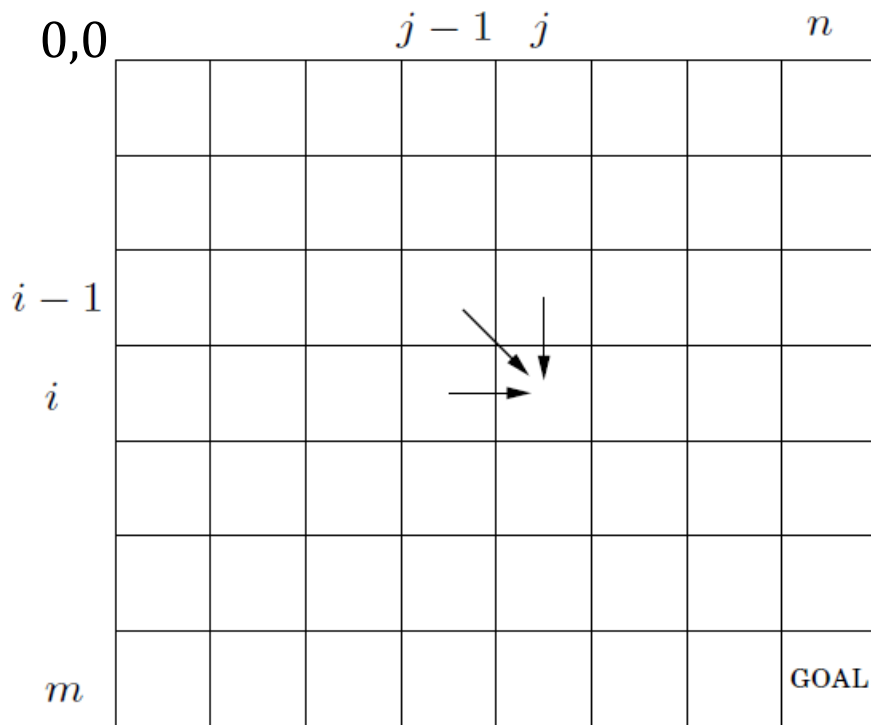
Can record

diff(i, j) is defined to be 0 if $x[i] = y[j]$ and 1 otherwise

11

the operation

$c > 1$



before $E(i, j)$

$E(i - 1, j)$

$E(i, j - 1)$

$E(i - 1, j - 1)$

Base cases:

$$E(i, 0) = i$$

$$E(0, j) = j$$

Computation can be done moving left to right row-wise or top to bottom column-wise.

Bottom up

Top down

Start from $E(m, n)$ and recursively call functions giving 'top', 'left' and 'top-left' values. [very similar]

```

for  $i = 0, 1, 2, \dots, m$ :
     $E(i, 0) = i$ 
for  $j = 1, 2, \dots, n$ :
     $E(0, j) = j$ 
for  $i = 1, 2, \dots, m$ :
    for  $j = 1, 2, \dots, n$ :
         $E(i, j) = \min\{E(i - 1, j) + 1, E(i, j - 1) + 1, E(i - 1, j - 1) + \text{diff}(i, j)\}$ 
return  $E(m, n)$ 

```

Run-time complexity: $O(mn)$

Memory: Storing the DP table!

$O(mn)$ $\longrightarrow$ Top down (nature of recursion)

Recursion stack! size: $O(m+n)$

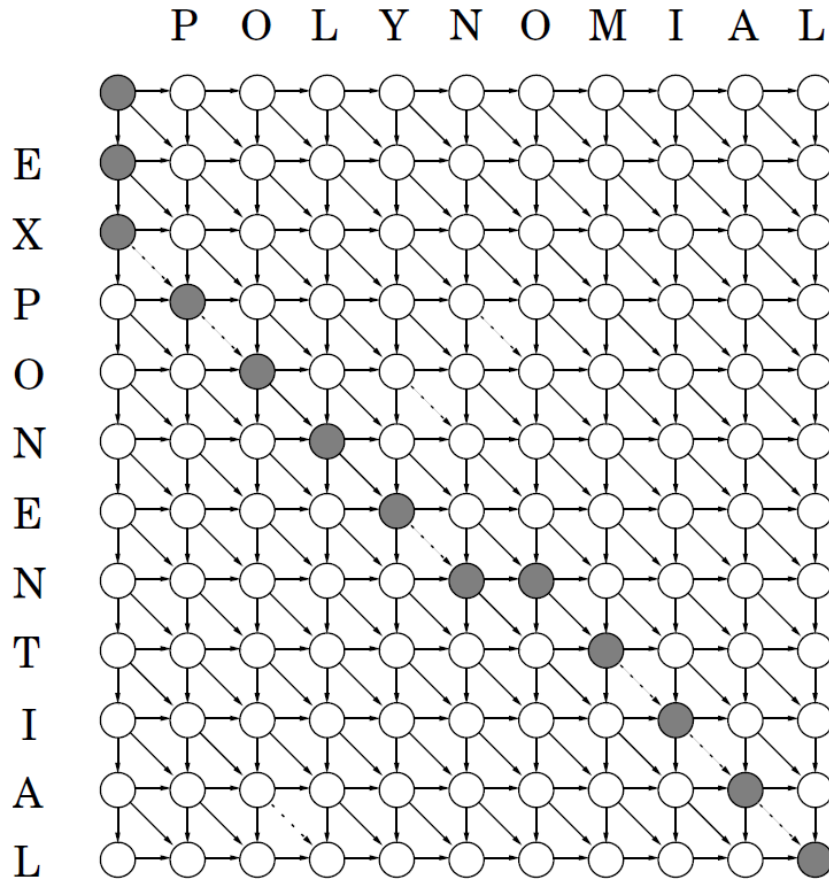
Bottom up:

Inside the loop we need to store only the 3 values from current & previous rows or current & previous columns!

$O(\min(m, n))$

The underlying DAG:

$(i-1, j) \rightarrow (i, j)$, $(i, j-1) \rightarrow (i, j)$, and $(i-1, j-1) \rightarrow (i, j)$



		P	O	L	Y	N	O	M	I	A	L
E	0	1	2	3	4	5	6	7	8	9	10
X	1	1	2	3	4	5	6	7	8	9	10
P	2	2	2	3	4	5	6	7	8	9	10
O	3	2	3	3	4	5	6	7	8	9	10
N	4	3	2	3	4	5	5	6	7	8	9
E	5	4	3	3	4	4	5	6	7	8	9
N	6	5	4	4	4	5	5	6	7	8	9
T	7	6	5	5	5	4	5	6	7	8	9
I	8	7	6	6	6	5	5	6	7	8	9
A	9	8	7	7	7	6	6	6	6	7	8
L	10	9	8	8	8	7	7	7	7	6	7
	11	10	9	8	9	8	8	8	8	7	6

edit distance 6

Shortest path in DAG with
edge weights 0 or 1 (or
different for different operations)

E X P O N E N — T I A L
— — P O L Y N O M I A L